Práctica 1 - AMC

Problema de los dos puntos más cercanos

Pablo Regidor García

ÍNDICE

1. Objetivo y descripción del problema
2. Estrategias implementadas
    2.1 Búsqueda exhaustiva
    2.2 Búsqueda exhaustiva con poda
    2.3 Divide y Vencerás
    2.4 Divide y Vencerás mejorado
3. Ordenación: HeapSort
    3.1 Ejemplos límite (mejor y peor caso)
    3.2 Estudio teórico: pseudocódigo y cálculo de la complejidad
4. Estudio experimental
    4.1 Metodología
    4.2 Caso medio (datos aleatorios distribuidos en x e y)
    4.3 Peor caso (todos los puntos en la misma vertical, misma x)
    4.4 Métrica de esfuerzo: distancias calculadas
5. Conclusiones

1. Objetivo y descripción del problema

En esta práctica hay que encontrar, dentro de un conjunto de puntos del plano, la pareja de puntos cuya distancia euclídea sea la menor de todas. Para resolverlo, he implementado cuatro estrategias: una búsqueda exhaustiva, que comprueba todos los pares posibles, esa misma búsqueda con poda, tras ordenar los puntos por la coordenada «x», el algoritmo Divide y Vencerás y una versión mejorada de este último. También he implementado la ordenación HeapSort, que usan las tres últimas estrategias.

El programa permite generar datasets aleatorios, ajustados al peor caso si se desea, cargar ficheros de TSPLIB, y muestra el resultado en una tabla con el par encontrado, su distancia, el número de distancias calculadas y el tiempo empleado, además de dibujar los puntos y la distancia mínima en un panel. Finalmente se comparan los tiempos de las cuatro estrategias con distintas tallas de datos para comprobar cuál funciona mejor.

2. Estrategias implementadas

2.1 Búsqueda exhaustiva

Es la versión más sencilla: se comprueba la distancia entre todos los pares de puntos posibles, que son $n \cdot (n-1)/2$, y se queda la menor comparación. Como se recorren todas las parejas, el coste es $O(n^2)$ siempre, sin importar cómo estén colocados los datos.

2.2 Búsqueda exhaustiva con poda

Es la misma idea pero ordenando primero los puntos por la coordenada «x». Una vez ordenados, en el bucle interno se deja de comparar en cuanto la diferencia entre las «x» de dos puntos es mayor que la mejor distancia calculada hasta ese momento: los puntos que quedan por la derecha están aún más lejos, por lo que se descartan todos sin calcular su distancia. Estas comparaciones de «x» no cuentan como distancias calculadas. El coste mínimo, es el de la ordenación, $O(n \cdot \log n)$, pero en la práctica el ahorro es mucho mayor.

2.3 Divide y Vencerás

Con los puntos ordenados por «x», se toma el punto central como línea vertical que parte el conjunto en dos mitades. El par más cercano o está en la izquierda, o en la derecha, o tiene un punto en cada lado. Los dos primeros casos se resuelven llamando recursivamente al algoritmo, y de los dos se queda la menor distancia. Para el caso de cruce basta mirar la franja central, formada por los puntos cuya «x» está

a menos de esa menor distancia de la línea; la búsqueda dentro de ella se hace con una pequeña poda por la «x». La recurrencia es O(n·log n).

2.4 Divide y Vencerás mejorado

El problema del algoritmo anterior aparece si todos los puntos tienen la misma «x», la franja central contiene a todos los puntos y la búsqueda dentro de ella vuelve a ser cuadrática. La mejora consiste en ordenar la franja por la coordenada «y» en cada llamada y comparar cada punto solo con sus 7 vecinos en esa lista.Esto funciona, debido a el argumento de empaquetado de la geometría computacional clásica, que nos dice que, usando la distancia mínima como referencia, en la franja ninguno de los puntos fuera de los 7 vecinos de nuestro punto actual, puede ser el más cercano. De este modo la franja deja de ser el punto débil y el peor caso queda neutralizado.

3. Ordenación: HeapSort

HeapSort es la ordenación que usan las tres últimas estrategias. Funciona en dos fases:

- **Fase 1:** Se construye un max-heap, un árbol donde cada posición «i», tiene sus hijos en las posiciones «2i+1» y «2i+2» y cada padre es mayor que sus hijos.
- **Fase 2:** Se intercambia la raíz con la última posición del heap y se reconstruye, dejando cada máximo colocado al final

Su complejidad es O(n·log n) en el mejor caso, en el medio y en el peor. Se eligió frente a QuickSort porque ese coste está garantizado siempre: el peor caso de QuickSort es $O(n^2)$, y el peor caso de esta práctica, todos los puntos con la misma «x», es precisamente el tipo de entrada que lo provoca. Así las diferencias de tiempo del estudio experimental se pueden atribuir a las estrategias de búsqueda y no a la ordenación.

3.1 Ejemplos límite (mejor y peor caso)

Imagen 3. Estados del HeapSort con datos aleatorios: estado inicial, max-heap construido (Fase 1) y lista ordenada (Fase 2).

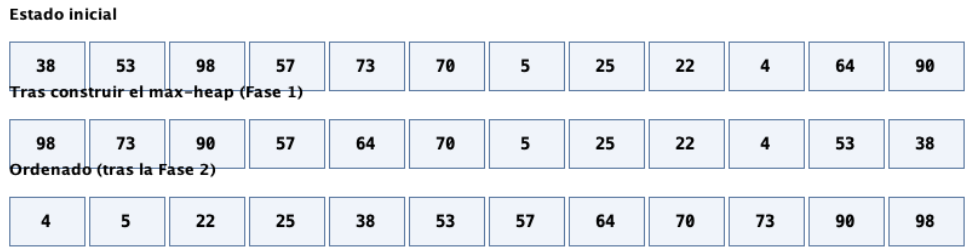
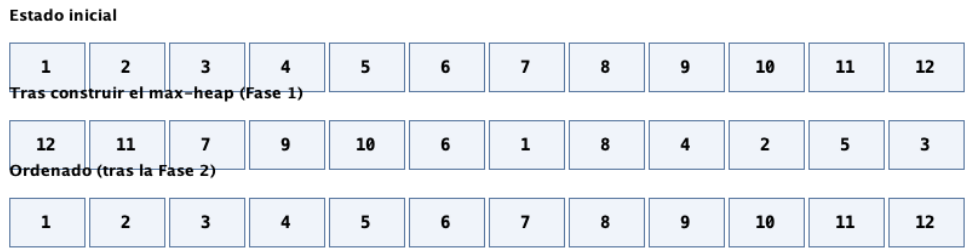


Imagen 4. Estados del HeapSort con datos ordenados ascendentes (caso clásico de estudio): mismas fases y O(n·log n) — la diferencia entre casos es constante, no asintótica.



3.2 Estudio teórico: pseudocódigo y cálculo de la complejidad

El pseudocódigo simplificado sobre el que se hace el análisis es el siguiente:

```
HEAPSORT(lista, n)
  PARA i = n/2 − 1 HASTA 0 HACER        // Fase 1: construir max-heap
      CRIBAR(lista, i, n)

  PARA fin = n−1 HASTA 1 HACER           // Fase 2: extraer el máximo
      intercambiar lista[0] con lista[fin]
      CRIBAR(lista, 0, fin)

CRIBAR(lista, i, fin)                     // hunde el elemento i
  MIENTRAS la posición i tenga hijos HACER
      hijo ← el mayor de 2i+1 y 2i+2      // COMPARE: operación básica
      SI lista[i] ≥ lista[hijo] ENTONCES salir
      intercambiar lista[i] con lista[hijo]
      i ← hijo
```

Para calcular el coste se usa como operación básica la comparación que la criba hace entre un elemento y su hijo mayor, porque decide si ese elemento sigue bajando en el árbol. En la primera fase, casi todos los padres están cerca de las hojas y descienden muy pocos niveles, y solo los pocos padres próximos a la raíz pueden bajar hasta el fondo: al sumar el trabajo de esos padres se obtiene un total que crece como n, de forma que construir el heap cuesta O(n). En la segunda fase se hacen n−1 extracciones, cada una con una criba de O(log n), de forma que la fase cuesta O(n·log n). Como la altura del heap no depende del orden de los datos, el mejor caso, el medio y el peor tienen el mismo orden, O(n·log n), esa garantía es la razón por la que se eligió HeapSort antes que QuickSort, cuyo peor caso cae a $O(n^2)$.

4. Estudio experimental

4.1 Metodología

El proceso utilizado para el estudio experimental ha sido el siguiente:

- Para cada talla (200, 500, 1000, 1500, 2000, 3000, 4000 y 5000) se generan datasets aleatorios y se ejecutan 10 experimentos por talla, promediando los tiempos de las cuatro estrategias.
- Cada estrategia recibe una copia nueva del dataset, de modo que las cuatro parten de idénticas condiciones iniciales.
- El tiempo mide todo el método, ordenación incluida, con System.nanoTime() y se informa en milisegundos. La resolución nanosegundo evita resultados de valor cero cuando la estrategia es más rápida que un milisegundo.
- Además del tiempo se cuenta el número de distancias de cada estrategia, las comparaciones de coordenadas para podar no cuentan.
- Las mismas tallas se repiten en el peor caso, para demostrar qué ocurre cuando la poda y la franja de Divide y Vencerás no pueden descartar nada.

4.2 Caso medio (datos aleatorios distribuidos en x e y)

Analizando los datos de la Tabla 1, la exhaustiva con poda es la estrategia más rápida en todas las tallas. La exhaustiva crece cuadráticamente, de 200 a 5000 puntos, su tiempo se multiplica casi por 9, su orden es $O(n^2)$, calculado de manera empírica. Divide y Vencerás y su versión mejorada crecen de forma suave y quedan por detrás de la poda, la mejorada paga la ordenación de la franja. Se confirma lo previsible, en condiciones normales gana la exhaustiva con poda. En las medias más pequeñas, por debajo del milisegundo, se observan leves variaciones que no crecen con la talla; se deben al ajuste interno de la máquina virtual Java durante la ejecución, y se reducen aumentando el número de experimentos.

Tabla 1. Tiempos medios en milisegundos — CASO MEDIO

Talla	Exhaustivo (ms)	ExhaustivoPoda (ms)	DivideVenceras (ms)	DyV Mejorado (ms)
200	0.9090	0.0606	0.0701	0.1021
500	0.1308	0.1033	0.3571	0.1637
1000	0.3618	0.1878	0.4104	1.0382
1500	0.8130	0.1772	0.0763	0.3610
2000	1.4446	0.0548	0.1016	0.2859
3000	2.9986	0.0398	0.1586	0.4627
4000	5.4429	0.0311	0.2174	0.6525

5000	8.1485	0.1321	0.2867	0.8288

Gráfica 1. Tiempos (escala logarítmica) por talla — caso medio

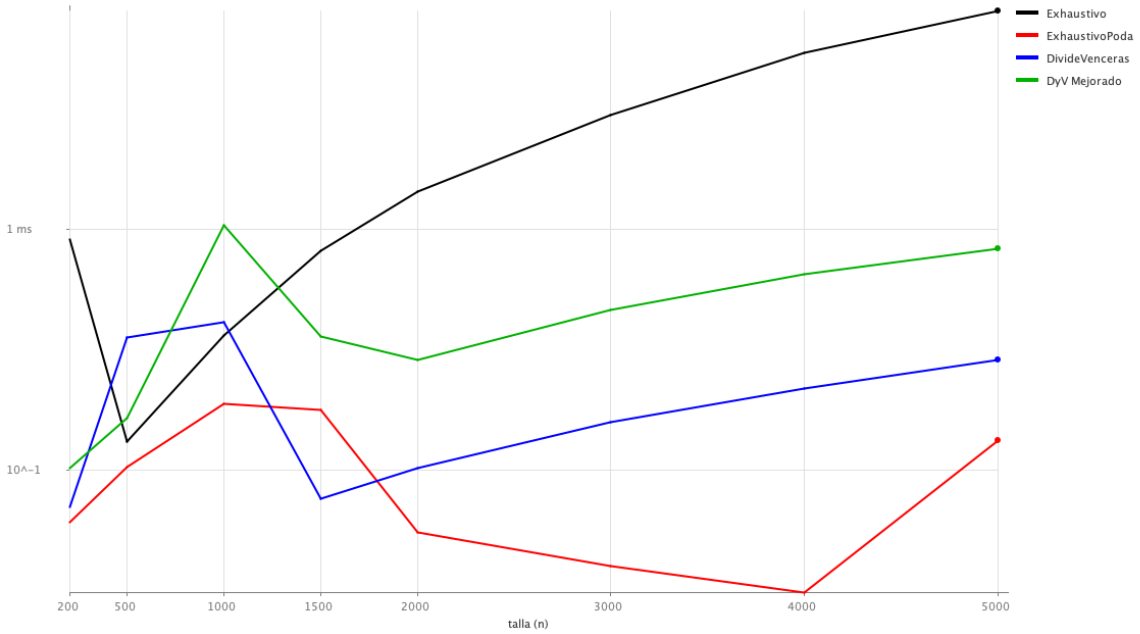
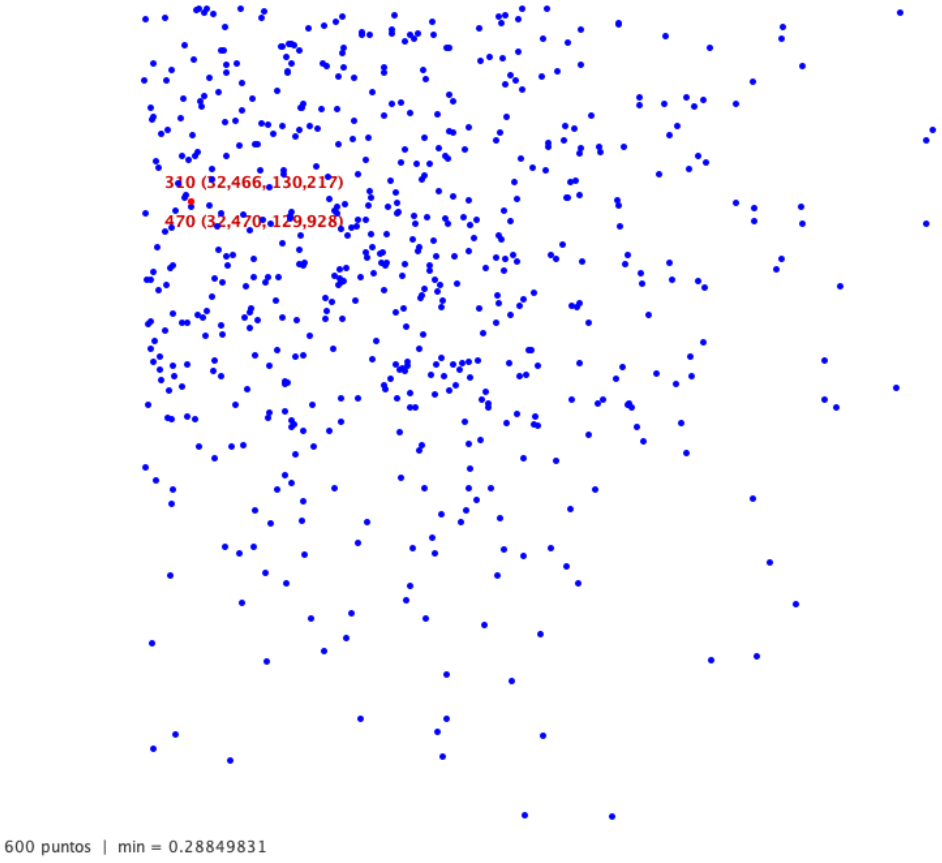


Imagen 6. Panel de solución en caso medio (600 puntos): dataset a escala y par más cercano en rojo.



4.3 Peor caso (todos los puntos en la misma vertical, misma x)

Según la Tabla 2, en el peor caso, los resultados se invierten respecto al caso medio. La exhaustiva con poda ya no descarta nada: todas las diferencias de «x» valen cero y su contador de distancias vuelve al valor de la exhaustiva, por lo que se comporta igual que ella pero sufriendo además la ordenación. Divide y Vencerás se ve afectado aún más, todos los puntos caen dentro de la franja y su coste vuelve a ser cuadrático, de modo que en la talla 5000, tarda más del doble que la exhaustiva. El más rápido es el Divide y Vencerás mejorado, con 4,77 ms, porque la ordenación de la franja por la «y» y los 7 vecinos limitan el trabajo dentro de ella. La imagen 8 muestra el dataset del peor caso.

Tabla 2. Tiempos medios en milisegundos — PEOR CASO

Talla	Exhaustivo (ms)	ExhaustivoPoda (ms)	DivideVenceras (ms)	DyV Mejorado (ms)
200	0.0149	0.0185	0.0347	0.0738
500	0.1000	0.1244	0.1856	0.2634
1000	0.3571	0.4309	0.6215	0.6225
1500	0.7463	0.9187	1.3940	1.0708
2000	1.3454	1.6521	2.4020	1.5180

3000	2.9594	3.6000	5.6005	2.4939
4000	5.2446	6.4478	9.9980	3.7745
5000	8.1597	10.3995	18.6734	4.7703

Gráfica 2. Tiempos (escala lineal) por talla — PEOR CASO

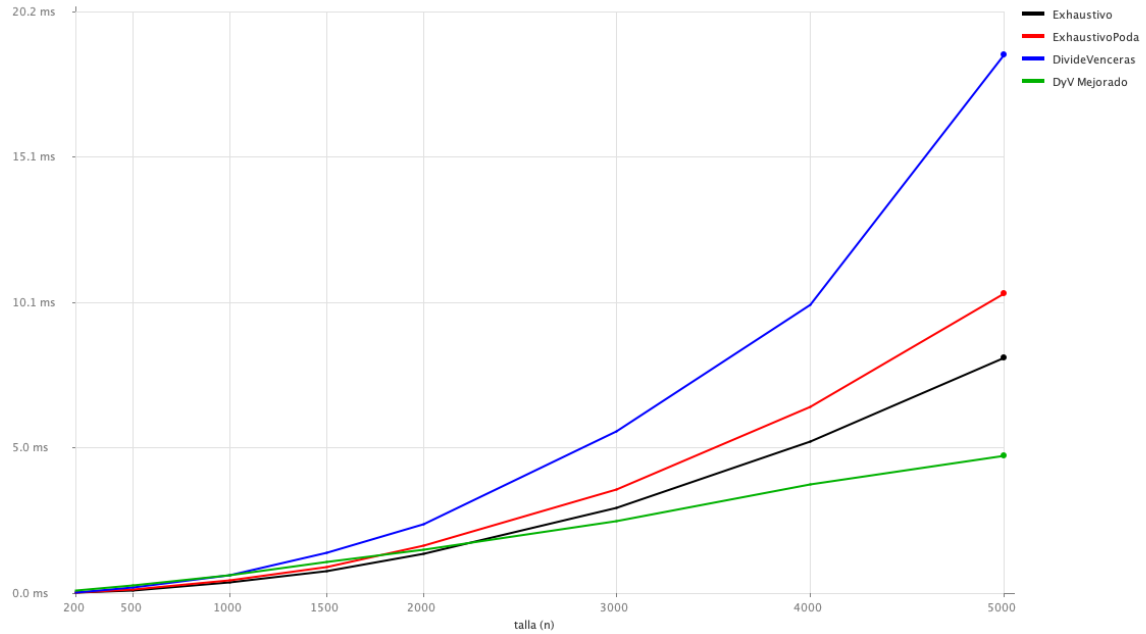
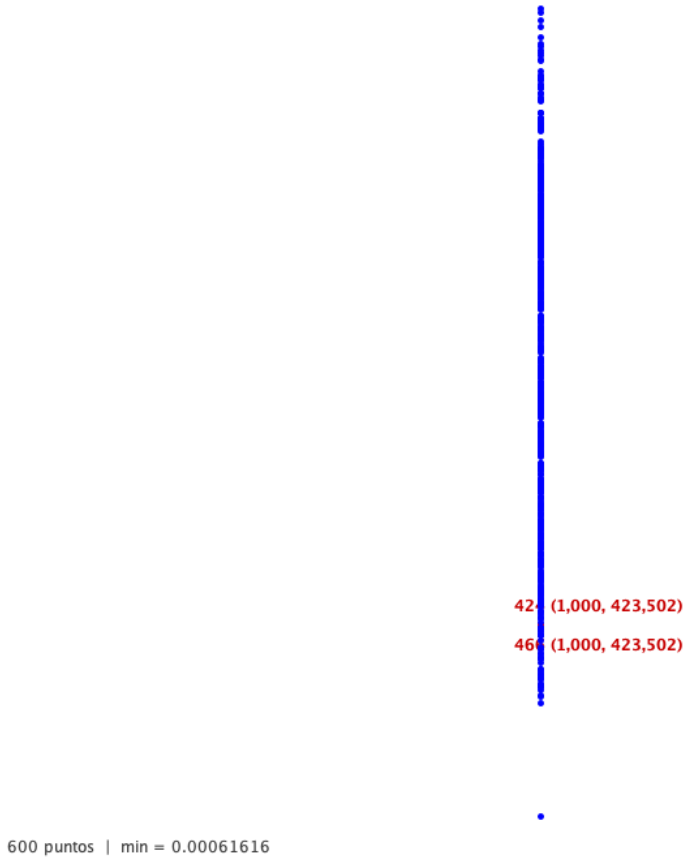


Imagen 8. Panel de solución en el peor caso: la misma vertical (misma x) — la imagen explica por qué la poda por x no puede descartar nada.



4.4 Métrica de esfuerzo: distancias calculadas

Para hacer la medición sin que afecten causas externas, se cuenta cuántas veces calcula cada estrategia la distancia entre un par de puntos y, al igual que en los tiempos, se anota la media sobre 10 datasets por talla. El proceso utilizado ha sido el siguiente:

- se ejecuta la opción 5 del programa, que cuenta las distancias calculadas por cada estrategia;
- se repite con tallas de 200 a 5000 puntos, en el caso medio y en el peor caso;

Tabla 3. Distancias euclídeas calculadas — CASO MEDIO

Talla	Exhaustivo	ExhaustivoPoda	DivideVenceras	DyV Mejorado
200	19,900	256	1,250	1,347
500	124,750	810	4,224	4,366
1000	499,500	1,583	11,031	10,661
1500	1,124,250	2,177	19,628	17,878

2000	1,999,000	2,559	27,835	24,710
3000	4,498,500	3,645	49,414	41,354
4000	7,998,000	6,108	74,211	59,240
5000	12,497,500	7,961	100,075	77,660

Según la Tabla 3, la exhaustiva calcula siempre n·(n−1)/2 distancias, porque comprueba todos los pares. La poda reduce el conteo de forma drástica: 7.961 distancias en 5000, unas 1.570 veces menos que la exhaustiva. Divide y Vencerás mantiene un conteo de 100.075 y la versión mejorada queda en 77.660, porque la ventana de 7 vecinos limita las comparaciones de cada punto en la franja.

Tabla 4. Distancias euclídeas calculadas — PEOR CASO

Talla	Exhaustivo	ExhaustivoPoda	DivideVenceras	DyV Mejorado
200	19,900	19,900	39,068	6,948
500	124,750	124,750	247,284	21,920
1000	499,500	499,500	994,068	50,812
1500	1,124,250	1,124,250	2,240,836	82,460
2000	1,999,000	1,999,000	3,987,136	115,596
3000	4,498,500	4,498,500	8,980,172	185,892
4000	7,998,000	7,998,000	15,972,272	259,164
5000	12,497,500	12,497,500	24,965,196	335,876

Según la Tabla 4, con todos los puntos en una misma vertical la poda pierde por completo su ventaja, su contador es idéntico al de la exhaustiva en todas las tallas. Divide y Vencerás, calcula 24.965.196 distancias, el doble que la exhaustiva, porque la franja concentra todo el problema en cada llamada recursiva. La versión mejorada se queda en 335.876, unas 74 veces menos que la versión normal. Esta métrica explica los tiempos de la Tabla 2, donde se calculan menos distancias, se tarda menos.

5. Conclusiones

Las conclusiones de la práctica son las siguientes:

- No existe una estrategia ganadora en todas las situaciones. La exhaustiva con poda es la más rápida cuando los puntos están repartidos por el plano, y el Divide y Vencerás mejorado lo es cuando los puntos están en la misma vertical; por eso, la mejora que se añade en la cuarta estrategia no es innecesaria, es la que decide el resultado en el peor caso.
- El tiempo de cada estrategia va ligado directamente al número de distancias que acaba calculando. En el peor caso, la poda deja de ahorrar distancias y el Divide y Vencerás normal llega a calcular el doble que la exhaustiva, con el correspondiente coste en tiempo.

- El detalle de ordenar la franja por la «y» cuesta un poco en el caso medio, pero convierte un algoritmo con riesgo cuadrático en uno estable para cualquier distribución de los datos: es el ejemplo más claro de cómo una pequeña variante puede cambiar el resultado.
- Parte del resultado experimental depende del método y no solo del algoritmo: usar el mismo dataset para las cuatro estrategias y medir con precisión suficiente cuando la estrategia tarda menos que un milisegundo son condiciones necesarias para que los tiempos sean comparables.